

Heap.cc

```
1  #include "heap.h"
2  #include "employee.h"
3
4  Employee heap[MAX_SIZE_HEAP];
5  int size = 0;
6
7  int Parent(int i) { return (i - 1) / 2; }
8  int LeftChild(int i) { return 2 * i + 1; }
9  int RightChild(int i) { return 2 * i + 2; }
10 bool IsHeapFull() { return size == MAX_SIZE_HEAP; }
11 bool IsHeapEmpty() { return size == 0; }
12
13 void HeapifyDown(int i, int _size) {
14     int current_max = SearchLargestElement(i, _size);
15
16     if (current_max != i) {
17         Employee temp = heap[i];
18         heap[i] = heap[current_max];
19         heap[current_max] = temp;
20
21         HeapifyDown(current_max, _size);
22     }
23 }
24
25 void SiftDown(Employee data, int i) {
26     if (i == 0) {
27         heap[0] = data;
28         return;
29     }
30
31     int parent = Parent(i);
32
33     if (heap[parent].salario < data.salario) {
34         heap[i] = heap[parent];
35         SiftDown(data, parent);
36     } else {
37         heap[i] = data;
38     }
39 }
40
41 int SearchLargestElement(int i, int _size) {
42     int left = LeftChild(i);
43     int right = RightChild(i);
```

} $O(1)$

$O(1)$

} $O(1)$

} $\max(0, \log_2 n)$

$O(\log_2 n)$ *

} $\max(0, 1) \rightarrow O(1)$

} $\max(0, \log_2 n)$

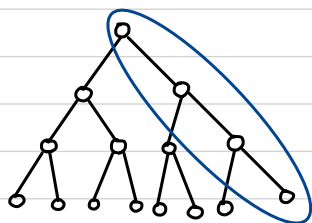
* Desarrollo en
la siguiente
página

Recurrencia Heapify Down

$$T(n) = T(n/2) + O(1)$$

Resolviendo con teorema maestro:

$$a = 1, b = 2, d = 0$$



Solo ve
un subarbol

Heapify Down solo recorre uno de los dos subarboles que es aproximadamente de tamaño $n/2$

$$\left. \begin{array}{l} \log_2(1) = 0 \\ 0 = d \end{array} \right\} \text{ caso 2} \quad \Downarrow \quad O(n^d \cdot \log_b n)$$

$$\begin{array}{l} T(n) = n^0 \cdot \log_2 n \\ T(n) = \log_2(n) \end{array}$$

Recurrencia Sift Down

$$T(n) = T(2n/3) + O(1)$$

La cantidad de datos en cada llamada recursiva es de $2n/3$ ya que tenemos un árbol desbalanceado, es decir un árbol que no tiene sus hojas llenas.

En el dato superior tenemos 1 nodo mientras que en las ramas tenemos $n-1$ nodos, por lo tanto en un lado donde hay más tenemos $\approx 2/3$ de los datos y en el otro $\approx 1/3$

Resolviendo con teorema maestro:

$$a = 1, b = 3/2, d = 0$$

$$\left. \begin{array}{l} \log_{3/2}(1) = 0 \\ 0 = d \end{array} \right\} \text{ caso 2}$$

$$T(n) = n^0 \cdot \log_{3/2} n$$

$$T(n) = \log_{3/2}(n)$$

$\log_{3/2} \wedge \log_2$ tienen la misma complejidad asintótica ya que

$$\log_{3/2} n = \frac{\log_2 n}{\log_2 \frac{3}{2}}$$

El factor $1/\log_{3/2}$ es solo una constante y en notación $O(n)$ las constantes se omiten

```

40 int SearchLargestElement(int i, int _size) {
41     int left = LeftChild(i);
42     int right = RightChild(i); }  $O(1)$ 
43     int current_max = i;
44
45     if (left < _size && heap[left].salario > heap[current_max].salario) { }  $\max(0,1) \rightarrow O(1)$ 
46         current_max = left;
47     }
48
49     if (right < _size && heap[right].salario > heap[current_max].salario) { }  $\max(0,1) \rightarrow O(1)$ 
50         current_max = right;
51     }
52
53     return current_max;  $O(1)$ 
54 }
55
56 bool RemovebyId(int id) {
57     if (IsHeapEmpty()) {
58         std::cerr << "Heap is empty\n"; }  $\max(0,1) \rightarrow O(1)$ 
59         return false;
60     }
61
62     heap[id] = heap[size - 1]; }  $O(1)$ 
63     size--;
64
65     HeapifyDown(id, size); }  $O(\log_2 n)$ 
66
67     return true;  $O(1)$ 
68 }

```

```

70 int SearchSmallestElement(int i, int _size) {
71     int left = LeftChild(i);
72     int right = RightChild(i); }  $O(1)$ 
73     int current_min = i;
74
75     if (left < _size && heap[left].salario < heap[current_min].salario) { }  $\max(0,1)$ 
76         current_min = left;
77     }
78
79     if (right < _size && heap[right].salario < heap[current_min].salario) { }  $\max(0,1)$ 
80         current_min = right;
81     }
82
83     return current_min;  $O(1)$ 
84 }

```

```

70 int SearchSmallestElement(int i, int _size) {
71     int left = LeftChild(i);
72     int right = RightChild(i); }  $O(1)$ 
73     int current_min = i;
74
75     if (left < _size && heap[left].salario < heap[current_min].salario) { }  $\max(0, 1)$ 
76         current_min = left;
77     }
78
79     if (right < _size && heap[right].salario < heap[current_min].salario) { }  $\max(0, 1)$ 
80         current_min = right;
81     }
82
83     return current_min;  $O(1)$ 
84 }
85
86 void InsertData(Employee data) {
87     if (IsHeapFull()) { }  $\max(0, 1) \rightarrow O(1)$ 
88         std::cerr << "Heap is full\n";
89         return;
90     }
91
92     heap[size] = data;  $O(1)$ 
93     SiftDown(data, size);  $\rightarrow O(\log_2 n)$ 
94     size++;  $O(1)$ 
95 }
96
97 Employee RemoveMin() {
98     if (IsHeapEmpty()) { }  $\max(0, 1) \rightarrow O(1)$ 
99         std::cerr << "Heap is empty\n";
100
101         Employee emptyEmployee;
102         emptyEmployee.salario = -1;
103         return emptyEmployee;
104     }
105
106     Employee EmployeeMin = heap[0];  $O(1)$ 
107
108     RemovebyId(0);  $O(\log_2 n)$ 
109
110     return EmployeeMin;  $O(1)$ 
111 }

```

```

113 void DisplayHeap(){
114     std::cout << "Contenido del heap:" << std::endl;  $O(1)$ 
115     for (int i = 0; i < size; i++)
116         std::cout << heap[i].salario << "\n"; }  $O(n)$ 
117 }
118
119 void OrderHeap() {
120     Employee sorted_employees[MAX_SIZE_HEAP]; }  $O(1)$ 
121     int original_size = size;
122
123     for (int i = 0; i < original_size; i++) {  $O(n)$ 
124         sorted_employees[i] = RemoveMin();  $O(\log n) \cdot O(n)$ 
125         std::cout << sorted_employees[i].nombre << " - " <<  $O(1)$ 
126     }
127 }
128
129 int SearchHeapByValue(float value) {
130     for (int i = 0; i < size; ++i) {
131         if (heap[i].salario == value) { }  $O(n)$ 
132             return i;
133     }
134 }
135 return -1;  $O(1)$ 
136 }

```

Recuento

Heapify Down $\rightarrow O(\log_2 n)$

Sift Down $\rightarrow O(\log_2 n)$

Insert Data $\rightarrow O(\log_2 n)$

Search Largest Element $\rightarrow O(1)$

Search Smallest Element $\rightarrow O(1)$

Remove Min $\rightarrow O(\log_2 n)$

Display Heap $\rightarrow O(n)$

Order Heap $\rightarrow O(n \log n)$

Search Heap By Value $\rightarrow O(n)$

Main.cc

```
1  #include <fstream>
2  #include "heap.h"
3  #include "employee.h"
4
5  void LoadDataFromFile(const char* filename) {
6      std::ifstream file(filename);
7
8      if (!file) {
9          std::cout << "Sorry: Could not open file " << filename << ".\n";
10         return;
11     }
12
13     Employee empleado; } O(1)
14     int count = 0;
15     while (file >> empleado.nombre >> empleado.apellido >> empleado.salario >> empleado.cargo) { } O(1)
16         InsertData(empleado); } O(1) · O(log n) → O(n log n)
17         count++; } O(1) · O(n)
18
19         if (IsHeapFull()) {
20             std::cout << "Error: Heap is full after " << count << " empleados.\n"; } O(1) · O(n)
21             break;
22         }
23     }
24
25     std::cout << "Loaded " << count << " employees from file " << filename << ".\n"; } O(1)
26     file.close();
27 }
28
29 int main(void) {
30     LoadDataFromFile("usuarios.txt"); } O(n log n)
31
32     int option = 0;
33     while (option ≠ 2) {
34         std::cout << "\nMENU\n";
35         std::cout << "1. Ordenar salarios\n"; } O(1)
36         std::cout << "2. Salir\n";
37         std::cout << "Opcion: ";
38         std::cin >> option;
39
40         switch (option) {
41             case 1:
42                 OrderHeap(); } O(n log n)
43                 break;
44             case 2:
45                 std::cout << "Saliendo ... \n"; } O(1)
46                 break;
47             default:
48                 std::cout << "Opcion invalida\n";
49         }
50     }
51
52     return 0;
53 }
54
```